
Approaches to reusability

“Follow the lead of hardware design! It is not right that every new development should start from scratch. There should be catalogs of software modules, as there are catalogs of VLSI devices: when we build a new system, we should be ordering components from these catalogs and combining them, rather than reinventing the wheel every time. We would write less software, and perhaps do a better job at that which we do get to write. Wouldn’t then some of the problems that everybody complains about — the high costs, the overruns, the lack of reliability — just go away? Why is it not so?”

You have probably heard remarks of this kind; perhaps you have uttered them yourself. As early as 1968, at the now famous NATO conference on software engineering, Doug McIlroy was advocating “mass-produced software components”. Reusability, as a dream, is not new.

It would be absurd to deny that some reuse occurs in software development. In fact one of the most impressive developments in the industry since the first edition of this book was published in 1988 has been the gradual emergence of reusable components, often modest individually but regularly gaining ground; they range from small modules meant to work with Microsoft’s Visual Basic (VBX) and OLE 2 (OCX, now ActiveX) to full libraries, also known as “frameworks”, for object-oriented environments.

Another exciting development is the growth of the Internet: the advent of a wired society has eased or in some cases removed some of the logistic obstacles to reuse which, only a few years ago, might have appeared almost insurmountable.

But this is only a beginning. We are far from McIlroy’s vision of turning software development into a component-based industry. The techniques of object-oriented software construction make it possible for the first time to envision a state of the discipline, in the not too distant future, in which this vision will have become the reality, for the greatest benefit not just of software developers but, more importantly, of those who need their products — quickly, and at a high level of quality.

In this chapter we will explore some of the issues that must be addressed for reusability to succeed on such a large scale. The resulting concepts will guide the discussion of object-oriented techniques throughout the rest of this book.

4.1 THE GOALS OF REUSABILITY

We should first understand why it is so important to improve software reusability. No need here for “motherhood and apple pie” arguments: as we will see, the most commonly touted benefits are not necessarily the most significant; by going beyond the obvious we can make sure that our quest for reuse will pursue the right targets, avoid mirages, and yield the highest return on our investment.

Expected benefits

From more reusable software you may expect improvements on the following fronts:

- **Timeliness** (in the sense defined in the discussion of software quality factors: speed of bringing projects to completion and products to market). By relying on existing components we have *less* software to develop and hence can build it faster.
- **Decreased maintenance effort.** If someone else is responsible for the software, that someone is also responsible for its future evolutions. This avoids the *competent developer's paradox*: the more you work, the more work you create for yourself as users of your products start asking you for new functionalities, ports to new platforms etc. (Other than relying on someone else to do the job, or retiring, the only solution to the competent software developer's paradox is to become an *incompetent* developer so that no one is interested in your products any more — not a solution promoted by this book.)
- **Reliability.** By relying on components from a reputed source, you have the guarantee, or at least the expectation, that their authors will have applied all the required care, including extensive testing and other validation techniques; not to mention the expectation, in most cases, that many other application developers will have had the opportunity to try these components before you, and to come across any remaining bugs. The assumption here is not necessarily that the component developers are any smarter than you are; simply that the components they build — be they graphics modules, database interfaces, sorting algorithms ... — are *their* official assignment, whereas for you they might just be a necessary but secondary chore for the attainment of *your* official goal of building an application system in your own area of development.
- **Efficiency.** The same factors that favor reusability incite the component developers to use the best possible algorithms and data structures known in their field of specialization, whereas in a large application project you can hardly expect to have an expert on board for *every* field touched on by the development. (Most people, when they think of the connection between reusability and efficiency, tend to see the reverse effect: the loss of fine-tuned optimizations that results from using general solutions. But this is a narrow view of efficiency: in a large project, you cannot realistically perform such optimizations on every piece of the development. You can, however, aim at the best possible solutions in your group's areas of excellence, and for the rest rely on someone else's expertise.)

This section is based on the more extensive discussion of management aspects of reuse in the book “Object Success” [M 1995].

- **Consistency.** There is no good library without a strict emphasis on regular, coherent design. If you start using such a library — in particular some of the best current object-oriented libraries — its style will start to influence, through a natural process of osmosis, the style of the software that you develop. This is a great boost to the quality of the software produced by an application group.
- **Investment.** Making software reusable is a way to preserve the know-how and inventions of the best developers; to turn a fragile resource into a permanent asset.

Many people, when they accept reusability as desirable, think only of the first argument on this list, improving productivity. But it is not necessarily the most important contribution of a reuse-based software process. The reliability benefit, for example, is just as significant. It is extremely difficult to build guaranteeably reusable software if every new development must independently validate every single piece of a possibly huge construction. By relying on components produced, in each area, by the best experts around, we can at last hope to build systems that we trust, because instead of redoing what thousands have done before us — and, most likely, running again into the mistakes that they made — we will concentrate on enforcing the reliability of our truly new contributions.

This argument does not just apply to reliability. The comment on efficiency was based on the same reasoning. In this respect we can see reusability as standing apart from the other quality factors studied in chapter 1: by enhancing it you have the potential of enhancing **almost all** of the other qualities. The reason is economic: if, instead of being developed for just one project, a software element has the potential of serving again and again for many projects, it becomes economically attractive to submit it to the best possible quality-enhancing techniques — such as formal verification, usually too demanding to be cost-effective for most projects but the most mission-critical ones, or extensive optimization, which in ordinary circumstances can often be dismissed as undue perfectionism. For reusable components, the reasoning changes dramatically; improve just one element, and thousands of developments may benefit.

This reasoning is of course not completely new; it is in part the transposition to software of ideas that have fundamentally affected other disciplines when they turned from individual craftsmanship to mass-production industry. A VLSI chip is more expensive to build than a run-of-the-mill special-purpose circuit, but if well done it will show up in countless systems and benefit their quality because of all the design work that went into it once and for all.

Reuse consumers, reuse producers

If you examined carefully the preceding list of arguments for reusability, you may have noted that it involves benefits of two kinds. The first four are benefits you will derive from basing your application developments on existing reusable components; the last one, from making your *own* software reusable. The next-to-last (consistency) is a little of both.

This distinction reflects the two aspects of reusability: the **consumer view**, enjoyed by application developers who can rely on components; and the **producer view**, available to groups that build reusability into their own developments.

In discussing reusability and reusability policies you should always make sure which one of these two views you have in mind. In particular, if your organization is new to reuse, remember that it is essentially impossible to start as a reuse producer. One often meets managers who think they can make development reusable overnight, and decree that no development shall henceforth be specific. (Often the injunction is to start developing “business objects” capturing the company’s application expertise, and ignore general-purpose components — algorithms, data structures, graphics, windowing and the like — since they are considered too “low-level” to yield the real benefits of reuse.) This is absurd: developing reusable components is a challenging discipline; the only known way to learn is to start by using, studying and imitating good existing components. Such an approach will yield immediate benefits as your developments will take advantage of these components, and it will start you, should you persist in your decision to become a producer too, on the right learning path.

Reuse Path principle

Be a reuse consumer before you try to be a reuse producer.

Here too “Object Success” explores the policy issues further.

4.2 WHAT SHOULD WE REUSE?

Convincing ourselves that Reusability Is Good was the easy part (although we needed to clarify *what* is really good about it). Now for the real challenge: how in the world are we going to get it?

The first question to ask is what exactly we should expect to reuse among the various levels that have been proposed and applied: reuse of personnel, of specifications, of designs, of “patterns”, of source code, of specified components, of abstracted modules.

Reuse of personnel

The most common source of reusability is the developers themselves. This form of reuse is widely practiced in the industry: by transferring software engineers from project to project, companies avoid losing know-how and ensure that previous experience benefits new developments.

This non-technical approach to reusability is obviously limited in scope, if only because of the high turnover in the software profession.

Reuse of designs and specifications

Occasionally you will encounter the argument that we should be reusing designs rather than actual software. The idea is that an organization should accumulate a repository of blueprints describing accepted design structures for the most common applications it develops. For example, a company that produces aircraft guidance systems will have a set of model designs summarizing its experience in this area; such documents describe module templates rather than actual modules.

This approach is essentially a more organized version of the previous one — reuse of know-how and experience. As the discussion of documentation has already suggested, the very notion of a design as an independent software product, having its own life separate from that of the corresponding implementation, seems dubious, since it is hard to guarantee that the design and the implementation will remain compatible throughout the evolution of a software system. So if you only reuse the design you run the risk of reusing incorrect or obsolete elements.

These comments are also applicable to a related form of reuse: reuse of specifications.

To a certain extent, one can view the progress of reusability in recent years, aided by progress in the spread of object technology and aiding it in return, as resulting in part from the downfall of the old idea, long popular in software engineering circles, that the only reuse worthy of interest is reuse of design and specification. A narrow form of that idea was the most effective obstacle to progress, since it meant that all attempts to build actual components could be dismissed as only addressing trivial needs and not touching the truly difficult aspects. It used to be the dominant view; then a combination of theoretical arguments (the arguments of object technology) and practical achievements (the appearance of successful reusable components) essentially managed to defeat it.

“Defeat” is perhaps too strong a term because, as often happens in such disputes, the result takes a little from both sides. The idea of reusing designs becomes much more interesting with an approach (such as the view of object technology developed in this book) which removes much of the gap between design and implementation. Then the difference between a module and a design for a module is one of degree, not of nature: a module design is simply a module of which some parts are not fully implemented; and a fully implemented module can also serve, thanks to abstraction tools, as a module design. With this approach the distinction between reusing modules (as discussed below) and reusing designs tends to fade away.

Design patterns

In the mid-nineteen-nineties the idea of *design patterns* started to attract considerable attention in object-oriented circles. Design patterns are architectural ideas applicable across a broad range of application domains; each pattern makes it possible to build a solution to a certain design issue.

Chapter 21 discusses the undoing pattern.

Here is a typical example, discussed in detail in a later chapter. The *issue*: how to provide an interactive system with a mechanism enabling its users to undo a previously executed command if they decide it was not appropriate, and to reexecute an undone command if they change their mind again. The *pattern*: use a class **COMMAND** with a precise structure (which we will study) and an associated “history list”. We will encounter many other design patterns.

[Gamma 1995]; see also [Pree 1994].

One of the reasons for the success of the design pattern idea is that it was more than an idea: the book that introduced the concept, and others that have followed, came with a catalog of directly applicable patterns which readers could learn and apply.

Design patterns have already made an important contribution to the development of object technology, and as new ones continue to be published they will help developers to

benefit from the experience of their elders and peers. How can the general idea contribute to reuse? Design patterns should not encourage a throwback to the “*all that counts is design reuse*” attitude mentioned earlier. A pattern that is *only* a book pattern, however elegant and general, is a pedagogical tool, not a reuse tool; after all, computing science students have for three decades been learning from their textbooks about relational query optimization, Gouraud shading, AVL trees, Hoare’s Quicksort and Dijkstra’s shortest path algorithm without anyone claiming that these techniques were breakthroughs in reusability. In a sense, the patterns developed in the past few years are only incremental additions to the software professional’s bag of standard tricks. In this view the new contribution is the patterns themselves, not the idea of pattern.

As most people who have looked carefully at the pattern work have recognized, such a view is too limited. There seems to be in the very notion of pattern a truly new contribution, even if it has not been fully understood yet. To go beyond their mere pedagogical value, patterns must go further. A successful pattern cannot just be a book description: it must be a **software component**, or a set of components. This goal may seem remote at first because many of the patterns are so general and abstract as to seem impossible to capture in actual software modules; but here the object-oriented method provides a radical contribution. Unlike earlier approaches, it will enable us to build reusable modules that still have replaceable, not completely frozen elements: modules that serve as general schemes (*patterns* is indeed the appropriate word) and can be adapted to various specific situations. This is the notion of *behavior class* (a more picturesque term is *programs with holes*); it is based on O-O techniques that we will study in later chapters, in particular the notion of deferred class. Combine this with the idea of groups of components intended to work together — often known as *frameworks* or more simply as *libraries* — and you get a remarkable way of reconciling reusability with adaptability. These techniques hold, for the pattern movement, the promise of exerting, beyond the new-bag-of-important-tricks effect, an in-depth influence on reusability practices.

See “*Programs with holes*”, page 506.

Reusability through the source code

Personnel, design and specification forms of reuse, useful as they may be, ignore a key goal of reusability. If we are to come up with the software equivalent of the reusable parts of older engineering disciplines, what we need to reuse is the actual stuff of which our products are made: executable software. None of the targets of reuse seen so far — people, designs, specifications — can qualify as the off-the-shelf components ready to be included in a new software product under development.

If what we need to reuse is software, in what form should we reuse it? The most natural answer is to use the software in its original form: source text. This approach has worked very well in some cases. Much of the Unix culture, for example, originally spread in universities and laboratories thanks to the on-line availability of the source code, enabling users to study, imitate and extend the system. This is also true of the Lisp world.

The economic and psychological impediments to source code dissemination limit the effect that this form of reuse can have in more traditional industrial environments. But a more serious limitation comes from two technical obstacles:

See also “*Formats for reusable component distribution*”, page 79 below.

- Identifying reusable software with reusable source removes information hiding. Yet no large-scale reuse is possible without a systematic effort to protect reusers from having to know the myriad details of reused elements.
- Developers of software distributed in source form may be tempted to violate modularity rules. Some parts may depend on others in a non-obvious way, violating the careful limitations which the discussion of modularity in the previous chapter imposed on inter-module communication. This often makes it difficult to reuse some elements of a complex system without having to reuse everything else.

A satisfactory form of reuse must remove these obstacles by supporting abstraction and providing a finer grain of reuse.

Reuse of abstracted modules

All the preceding approaches, although of limited applicability, highlight important aspects of the reusability problem:

- Personnel reusability is necessary if not sufficient. The best reusable components are useless without well-trained developers, who have acquired sufficient experience to recognize a situation in which existing components may provide help.
- Design reusability emphasizes the need for reusable components to be of sufficiently high conceptual level and generality — not just ready-made solutions to special problems. The classes which we will encounter in object technology may be viewed as design modules as well as implementation modules.
- Source code reusability serves as a reminder that software is in the end defined by program texts. A successful reusability policy must produce reusable program elements.

The discussion of source code reusability also helps narrow down our search for the proper units of reuse. A basic reusable component should be a software element. (From there we can of course go to *collections* of software elements.) That element should be a *module* of reasonable size, satisfying the modularity requirements of the previous chapter; in particular, its relations to other software, if any, should be severely limited to facilitate independent reuse. The information describing the module's capabilities, and serving as primary documentation for reusers or prospective reusers, should be *abstract*: rather than describing all the details of the module (as with source code), it should, in accordance with the principle of Information Hiding, highlight the properties relevant to clients.

The term **abstracted module** will serve as a name for such units of reuse, consisting of directly usable software, available to the outside world through a description which contains only a subset of each unit's properties.

The rest of part B of this book is devoted to devising the precise form of such abstracted modules; part C will then explore their properties.

More on distribution formats below.

The emphasis on abstraction, and the rejection of source code as the vehicle for reuse, do not necessarily prohibit *distributing* modules in source form. The contradiction is only apparent: what is at stake in the present discussion is not how we will deliver modules to their reusers, but what they will use as the primary source of information about them. It may be acceptable for a module to be distributed in source form but reused on the basis of an abstract interface description.